AUDIT

`.Translate(` Call-Site Argument-Contract Audit (§11.4.118 discovery-pressure)

Revision: 1 **Last modified:** 2026-06-16T15:33:53Z **Mode:** READ-ONLY, NON-COMMITTING (per dispatch; sole committer is a separate agent). **Method:** superpowers:systematic-debugging + §11.4.6 (FACT-only) + §11.4.118. **Probe:** one temporary `_test.go` written under `pkg/translator/llm/`, run, then DELETED (confirmed no residue, nothing staged/committed).

---

0. The three distinct `Translate` interfaces (FACT)

The bug class is interface-specific. There are THREE unrelated `Translate` signatures; only ONE carries the bug class.

#	Interface	Signature	Semantics
I1	`LLMClient` (`pkg/translator/llm/llm.go:225`)	`Translate(ctx, text, prompt)`	RAW provider client. **OpenAI/ DeepSeek/Cloudflare/ Anthropic/Qwen/Zhipu/ Replicate send ONLY the 2nd arg `prompt` as the user message and IGNORE `text`.** Gemini is the exception (uses BOTH via `buildPrompt(text,prompt)`).
I2	`Translator` (`pkg/translator/translator.go:38`)	`Translate(ctx, text, context)`	HIGH-LEVEL. `*LLMTranslator` (llm.go:352) **composes a real prompt** from `text`+`context` and passes it as the 2nd arg to the raw client (llm.go:388). `clientTranslator` (bridge.go:554) delegates **VERBATIM** to the raw client — NO composition.
I3			Unrelated signature. N/A.

#	Interface	Signature	Semantics
	gRPC `Translator` (`pkg/grpc/server.go:78`)	`Translate(ctx, *proto.TranslationRequest, *EventBus)`	

Decisive distinction (FACT): an I2 call reaching a `*LLMTranslator` is SAFE even with an empty 2nd arg (the prompt is composed from the text). An I2 call reaching a `clientTranslator` is EXACTLY equivalent to a raw I1 call — so empty/whitespace 2nd arg → empty user message (empty-payload bug), and content-in-1st + non-empty-but-wrong 2nd arg → wrong-content bug (the 1st arg is dropped).

`clientTranslator` is produced ONLY by `Bridge.ProviderDiverseTranslators` / `EnsembleFactory` (bridge.go:534) and is **wired in production** at `cmd/server/main.go:96`, `cmd/cli/main.go:362`, and `cmd/markdown-translator/main.go:195` (preparation/verification ensemble seams). `bridge.clientBuild` builds ONLY `NewOpenAIClient` (bridge.go:314), so every clientTranslator's underlying client is an OpenAI-compatible (2nd-arg-only) client.

1. Wire-level probe results (FACT — captured, no API key)

Raw `OpenAIClient`/DeepSeek (== the verbatim clientTranslator) against an httptest server that records the user-message content:

```
(A) Translate(text="REAL CHAPTER TEXT", prompt="")        ->
userMsg=""            (EMPTY → empty-payload bug)
(B) Translate(text="VERIFICATION PROMPT", prompt="Chapter 3") ->
userMsg="Chapter 3"   (1st arg dropped → wrong-content bug)
(C) Translate(text="", prompt="Translate: REAL CHAPTER TEXT") ->
userMsg="Translate: REAL CHAPTER TEXT" (correct)
LLMTranslator.Translate(text="REAL CHAPTER TEXT", ctx="")  ->
userMsg(len=762)="You are a professional literary translator..."
(SAFE — composed, contains the text)
```

(A) is `cmd/markdown-translator/main.go:244` & `preparation/coordinator.go:290`-class. (B) is `verification/polisher.go:563` (ensemble path). (C) is the correct convention.

---

2. Full call-site table

Receiver-reached column states the CONCRETE type the call lands on in the production wiring. Verdict is w.r.t. the empty/wrong user-message bug class.

#	file:line	call (args)	receiver reached
1	`cmd/markdown-translator/main.go:244`	`LLMProvider.Translate(ctx, text, "")`	I1 raw client (`bridge.BestClie` OpenAIClient)

#	file:line	call (args)	receiver reached
2	pkg/preparation/coordinator.go:290	provider.Translate(ctx, prompt, "")	clientTranslator (ensemble path, mar translator:195)
3	pkg/preparation/coordinator.go:361	provider.Translate(ctx, prompt, "")	same as #2
4	pkg/preparation/coordinator.go:424	provider.Translate(ctx, prompt, "")	same as #2
5	pkg/verification/polisher.go:563	translator.Translate(ctx, prompt, location)	clientTranslator (ensemble path) OR *LLMTranslator (d
6	pkg/api/batch_handlers.go:146	trans.Translate(ctx, req.Text, "")	*LLMTranslator
7	pkg/api/handler.go:833	dt.Translate(ctx, text, contextHint)	distributedTrans → dm.TranslateDist
8	pkg/coordination/multi_llm.go:445	instance.Translator.Translate(ctx, text, contextHint)	*LLMTranslator (d multi_llm.go:210) C clientTranslator (ensemble, cli/server

#	file:line	call (args)	receiver reached
9	pkg/coordination/multi_llm.go:529	inst.Translator.Translate(ctx, text, contextHint)	same as #8
10	pkg/distributed/coordinator.go:308	localTranslator.Translate(ctx, text, contextHint)	∗LLMTranslator (llm.NewLLMTrans coordinator.go:297/3
11	pkg/api/server.go:196	s.translator.Translate(ctx, req.Text, "ru->sr")	unconfirmed (SetTranslator, server.go:120)
12	pkg/batch/processor.go:110	Translator.Translate(ctx, InputString, "")	∗LLMTranslator (A batch_handlers.go:2
13	pkg/batch/processor.go:163	Translator.Translate(ctx, string(data), "")	same as #12
14	pkg/batch/processor.go:513	Translator.Translate(ctx, text, "")	same as #12
15	pkg/api/server.go:196	(covered as #11)	—
16	pkg/grpc/server.go:500	s.translator.Translate(session.Ctx, session.Request, session.EventBus)	I3 gRPC core transla
17	pkg/markdown/simple_workflow.go:92	LLMProvider.Translate(ctx, text, prompt)	I1 raw client
18	pkg/translator/llm/llm.go:388	lt.client.Translate(ctx, text, prompt)	I1 raw client
19	pkg/translator/llm/llm.go:414	lt.client.Translate(ctx, chunk, chunkPrompt)	I1 raw client
20	pkg/translator/llm/llm.go:566	lt.Translate(ctx, text, contextStr)	self (∗LLMTranslat
21	pkg/translator/llm/qwen.go:363	c.Translate(ctx, text, prompt)	self (QwenClient)
22	pkg/verification/notes.go:166	nt.translator.Translate(ctx, prompt, location)	same class as #5 (po

#	file:line	call (args)	receiver reached
23	pkg/bridge/bridge.go:369	client.Translate(ctx, "", full)	I1 raw client
24	pkg/bridge/bridge.go:555	c.client.Translate(ctx, text, contextStr)	I1 raw client (inside clientTranslator
25	pkg/bridge/bridge.go:571	c.Translate(ctx, text, contextStr)	self (clientTranslator

---

3. Counts

- **Confirmed BROKEN (FACT, wire-proven bug class, reachable in a real production wiring): 5 sites**
  - empty-payload (empty user message): #1 `markdown-translator/main.go:244`, #2/#3/#4 `preparation/coordinator.go:290/361/424` (ensemble path).
  - wrong-content (1st-arg dropped, prompt replaced by location): #5 `verification/polisher.go:563` (ensemble path).
- **NEEDS-RUNTIME-CONFIRM: 4 sites** — #8/#9 `coordination/multi_llm.go:445/529` (broken iff ensemble + empty/garbage contextHint), #11 `api/server.go:196` (receiver type unproven), #22 `verification/notes.go:166` (same wrong-content pattern as polisher, injector untraced).
- **SAFE:** #6, #7, #10, #12-14, #17-21, #23 (and conduits #24/#25).
- **N/A:** #16 (gRPC I3), #21 (pass-through retry).

Probes that would settle the NEEDS-RUNTIME items (§11.4.6):

- #8/#9: instrument the ebook→`MultiLLMTranslatorWrapper.Translate` call and capture the `context` arg actually passed during a real chapter translation.
- #11: `grep` the running binary's wiring for `Server.SetTranslator(...)`; if no production caller, the path is dead (investigate per §11.4.124 before claiming).
- #22: trace the constructor that populates `notes` translator (built-in `NewLLMTranslator` ⇒ SAFE; ensemble factory ⇒ wrong-content).

---

4. Defense-in-depth fix coverage (FACT)

Proposed fix in `OpenAIClient.Translate` (covers DeepSeek via embedding, and the entire `clientTranslator`/`BestClient` path since `bridge.clientBuild` builds ONLY `NewOpenAIClient`):

1. when `prompt == ""` → use `text` as the user message;
2. refuse an empty/whitespace user message → return an explicit error (§11.4.69).

Trigger set (the ONLY sites where clause 1 changes behavior) = the 8 calls with a literal "" 2nd arg (grep-proven): markdown-translator:244; preparation 290/361/424; batch processor 110/163/513; batch_handlers:146.

- **#1 markdown-translator:244** — fallback sends the real chapter text → **FIXED** (probe: empty→text).
- **#2/#3/#4 preparation ensemble** — `prompt` (built analysis prompt) in 1st arg, fallback sends it → **FIXED** (the analysis prompt becomes the user message — the intended behavior).
- **batch #12-14 & #6 batch_handlers** — reach `*LLMTranslator`, which NEVER calls the raw client with an empty 2nd arg (it composes), so clause 1 never fires there → **UNCHANGED / still SAFE**. No SAFE site relies on the current ignore-1st-arg behavior to be broken — verified: every content-in-2nd-arg SAFE site (#17, #18, #19, #23) passes a NON-empty 2nd arg, so clause 1 (`prompt==""`) is inert for them, and clause 2's empty-message guard never fires (their messages are non-empty).

Coverage verdict:

- The fix **FULLY covers all 4 EMPTY-PAYLOAD broken sites (#1-#4)** without breaking any SAFE site. **FACT.**
- The fix **does NOT cover the WRONG-CONTENT sites (#5 polisher, #22 notes)**: their 2nd arg (`location`) is NON-EMPTY ("Chapter N"), so clause 1's `prompt==""` fallback never fires and clause 2's empty-guard never triggers — the raw client still sends `location` and silently drops the verification prompt. Probe (B) proves this. These need a SEPARATE fix (either route the polisher/notes calls through a prompt-composing translator, or have the verification path pass content in the 2nd arg per the bridge.go:369 convention).

Recommendation: ship the `OpenAIClient.Translate` defense-in-depth fix (it converts every latent empty-payload occurrence into either a correct translation or a loud error — covers #1-#4 and any future empty-2nd-arg caller), AND open a separate item for the wrong-content class at the verification seam (#5/#22) — the empty-guard cannot catch a non-empty-but-wrong user message.

---

5. §11.4.6 honesty boundary

- PROVEN as FACT (wire repro): the I1 raw / verbatim-clientTranslator contract; the empty-payload bug at #1 (also FINDING.md) and #2-#4; the wrong-content bug at #5; the `*LLMTranslator` composes-and-is-SAFE behavior; and that the proposed fix covers #1-#4 without breaking SAFE sites but does NOT cover #5/#22.
- UNCONFIRMED (NEEDS-RUNTIME-CONFIRM, listed in §3): #8/#9 contextHint value at runtime; #11 server.translator concrete type / whether the path is live; #22 notes translator injector. Each marked, not guessed.
- The ensemble-path verdicts (#2-#5) are FACT for the ensemble wiring (markdown-translator/cli/server use `b.EnsembleFactory`); the built-in default path for those same sites is independently SAFE.
- No files staged or committed. The probe `_test.go` was deleted; `git status` clean of any probe residue.